Construction Planning Assistant Agent

Course Name: Agentic Ai

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vivek Sharma	EN22ME304116
2.	Jayesh Chouhan	EN23MEEL4001
3.	Sakina Modi	EN22EL301044
4.	Vinay Birla	EN22CS3011083
5.	Dev Singh Ghuraiya	EN22ME304022

Group Name:05D10

Project Number:DO-18

Industry Mentor Name:

University Mentor Name: Maya Yadav

Academic Year:2025-26

Table of Contents

1. Introduction

2. Problem Statement & Objectives
   2.1 Problem Statement
   2.2 Project Objectives
   2.3 Scope of the Project

3. Proposed Solution
   3.1 Key Features
   3.2 Overall Architecture / Workflow
   3.3 Tools & Technologies Used

4. Results & Output
   4.1 Screenshots / Outputs
   4.2 Reports
   4.3 Key Outcomes

5. Conclusion

6. Future Scope & Enhancements

1. Introduction

This project presents the Construction Planning Assistant Agent, an AI-powered agentic system named Arch, built to automate and streamline construction project planning. Arch accepts a high-level natural language goal from the user and autonomously decomposes it into a structured, phase-by-phase execution plan by reasoning over the task, calling specialized tools, observing results, and iterating — all in real time.

The system demonstrates the practical application of Agentic AI — specifically the ReAct (Reason + Act) pattern — where an LLM does not simply generate text but actively decides which tools to call, interprets their outputs, and uses that information to produce grounded, data-backed plans.

---

2. Problem Statement & Objectives

2.1 Problem Statement

Engineer a sophisticated 'Planner Agent' designed to orchestrate complex construction tasks through a multi-step reasoning loop. Upon receiving a high-level goal (e.g., 'Permit Requirements'), the agent autonomously decomposes the objective into actionable steps, validates resource availability through mock interface tools, and generates a detailed execution schedule. This implementation highlights the agent's ability to handle dependencies and optimize task sequences in a dynamic environment..

Problem Overview

- Multiple dependent phases (site → foundation → framing → finish)
- Each phase needs validation (materials, labor, permits, time)
- Manual tools lack dependency reasoning
- AI chatbots may generate unverified (hallucinated) plans
- No lightweight open-source tool combining LLM + real data

Core Challenge

- Build an agent that understands goals, gathers data, uses tools correctly, and generates reliable structured plans

2.2 Project Objectives

- Build an AI agent that decomposes construction goals into phases
- Implement a multi-step reasoning loop (ReAct pattern)
- Integrate tool-calling for materials, labor, permits, and scheduling
- Ensure the agent uses only tool-returned data (no hallucination)
- Stream agent reasoning and tool execution live to the UI
- Build a professional web interface with real-time feedback
- Persist all plans with structured phase data for history review
- Provide a Gantt chart and phase breakdown dashboard

2.3 Scope of the Project

In Scope:

- Natural language construction goal input
- Autonomous phase decomposition into 4 standard phases: Site Preparation, Foundation, Framing, Interior Finish
- Tool-calling for 4 data points per phase: material availability, labor availability, permit status, phase duration
- Real-time streaming of agent activity via Server-Sent Events
- Web-based chat interface with result dashboard (metrics, phase table, Gantt chart, agent log)
- Persistent plan history with structured JSON storage
- Selectable tool pills allowing users to control which tools the agent uses

Out of Scope:

- Real integration with construction management databases or government permit APIs (tools are simulated)
- Multi-user authentication and authorization
- Cost estimation with real market pricing
- Mobile application
- Integration with project management tools (Jira, MS Project)
- Real-time collaboration between multiple planners

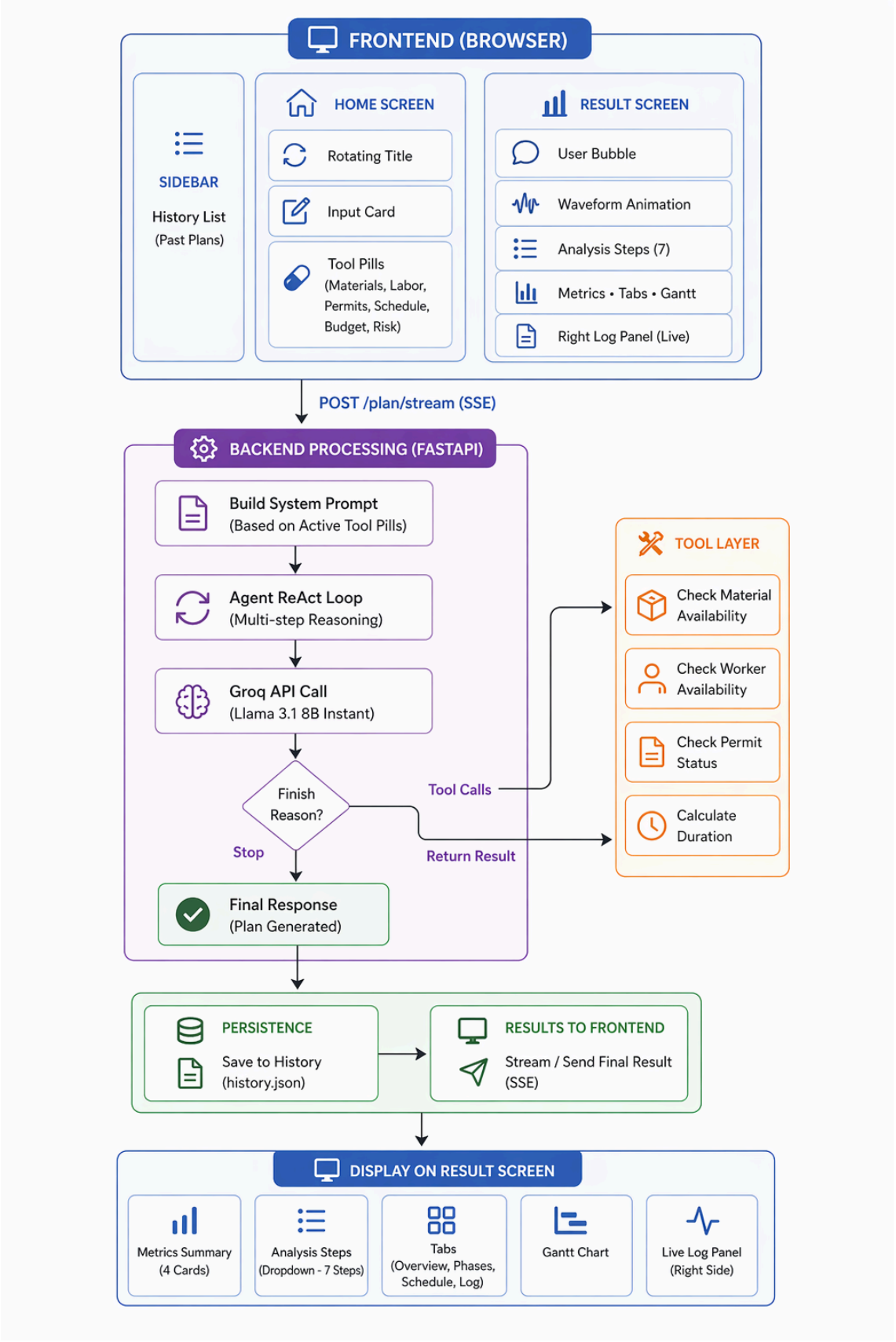
---

3. Proposed Solution

3.1 Key Features

- **Agentic ReAct Loop** – Multi-step reasoning loop where the agent selects tools, executes them, and iterates before generating the final plan
- **Dynamic Tool Selection** – Users can enable/disable tools (Materials, Labor, Permits, Schedule, Budget, Risk) to control output and reduce token usage
- **Real-Time Streaming** – Agent actions stream live via SSE, showing tool calls and results before the final output
- **Anti-Hallucination Prompt** – Ensures the agent uses only tool-returned data, not invented values
- **Result Dashboard** – Displays metrics, structured phases, Gantt chart, and full plan view
- **Live Agent Log Panel** – Real-time, color-coded logs of agent steps (THINK, CALL, OBS, DONE)
- **Persistent History** – Saves all plans with structured data for easy access and review

3.2 Overall Architecture / Workflow



Total Layers: 4

Frontend (Browser)

- Handles user interaction (input, tool selection, viewing results)
- Displays real-time agent activity and final plan (metrics, Gantt chart, logs)
- Provides history access via sidebar

Backend Processing (FastAPI)

- Builds system prompt based on selected tools
- Runs the Agentic ReAct loop for multi-step reasoning
- Communicates with LLM and controls execution flow

Tool Layer

- **Executes domain-specific functions:**
  - Material availability
  - Worker availability
  - Permit status
  - Duration calculation

- **Returns real, structured data to the agent**

Persistence & Output Layer

- Saves generated plans to history (`history.json`)
- Streams results back to frontend using SSE
- Enables result visualization and reuse

Agent Execution Workflow:

Step 1: User enters goal and selects tools, then clicks submit.

Step 2: Request is sent to POST /plan/stream and system prompt is built.

Step 3: LLM decides the sequence of tool calls.

Step 4: For each phase, tools are called (materials, workers, permits, duration) and streamed via SSE.

Step 5: LLM uses all results to generate the final plan.

Step 6: Final response is streamed and saved to history.json.

Step 7: Frontend renders the dashboard.

3.3 Tools & Technologies Used

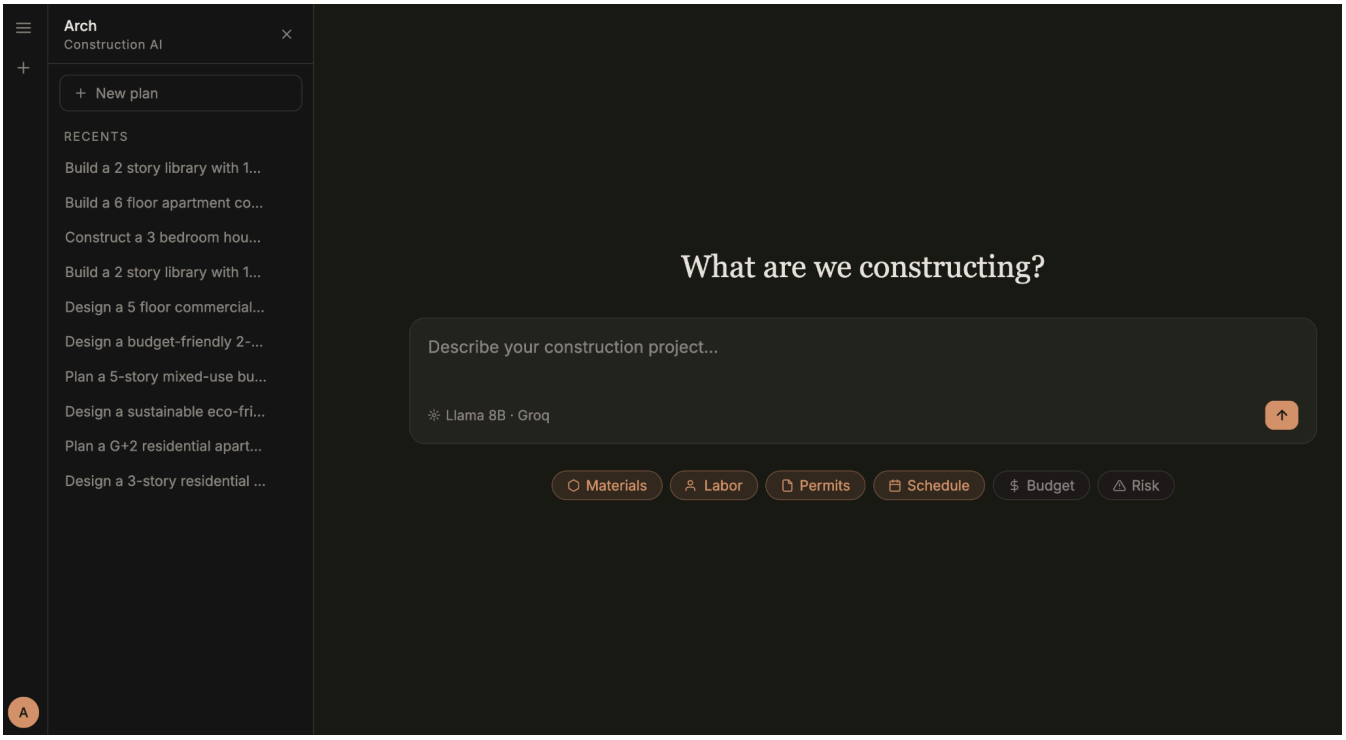
Component	Technology
API/Backend	Python, FastAPI
API Server	Uvicorn
Runtime AI	Groq + Llama 3.1
Frontend Logic	HTML/CSS/JS
UI/Real-time	Stream SSE
Storage	JSON Data
Agent Pattern	ReAct Flow

---

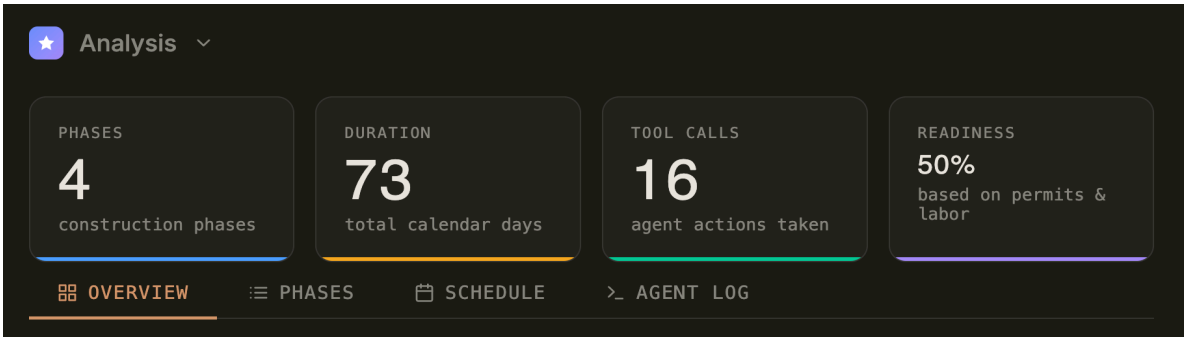
4. Results & Output

4.1 Screenshots / Outputs

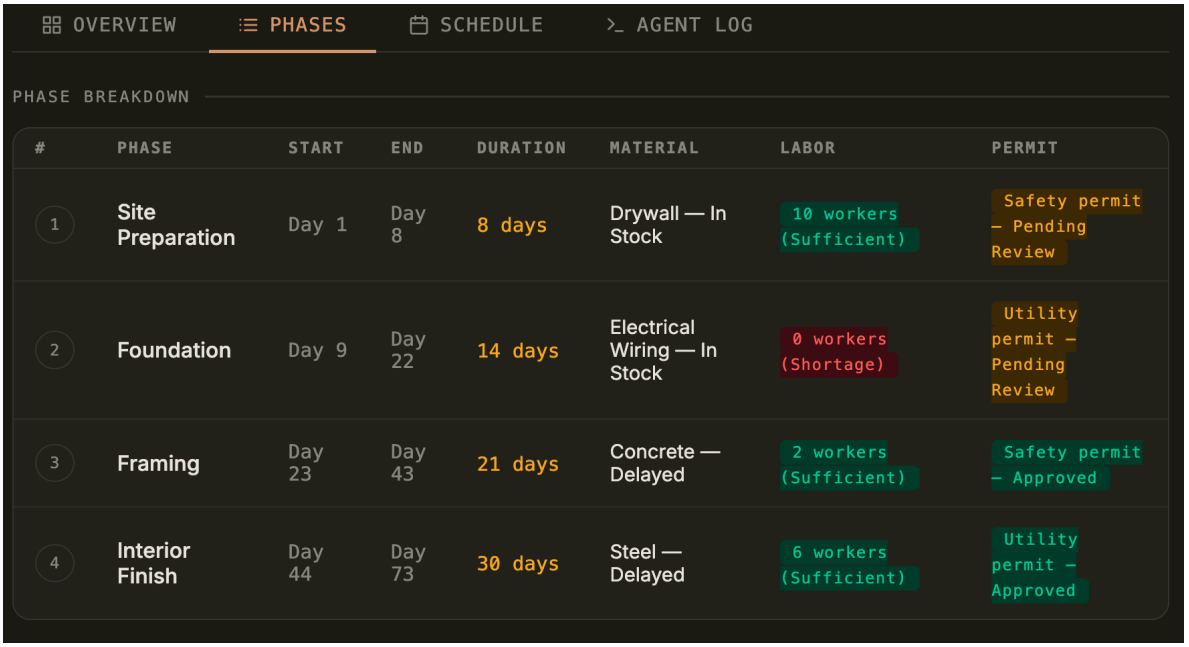
Home Screen



Result Screen — Metrics + Tabs

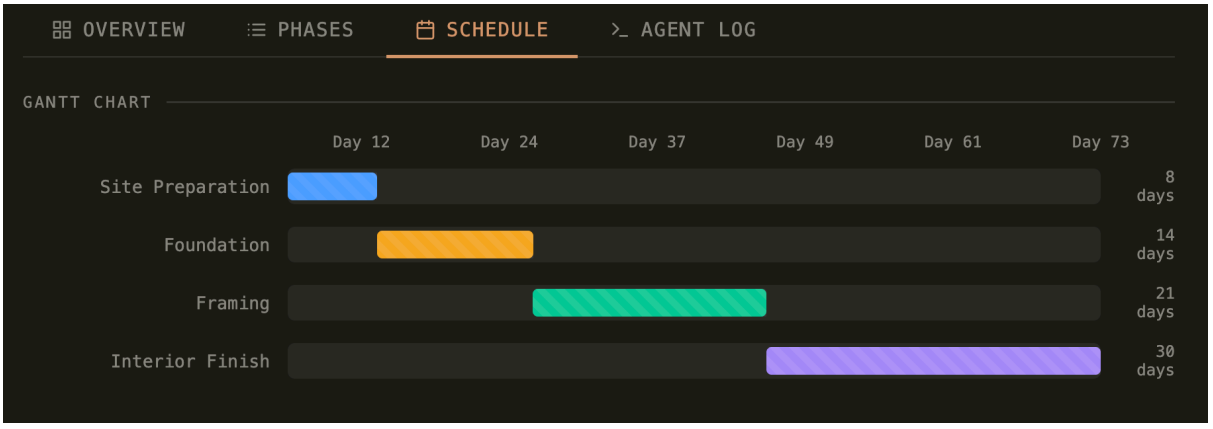


Phases Tab — Sample Output

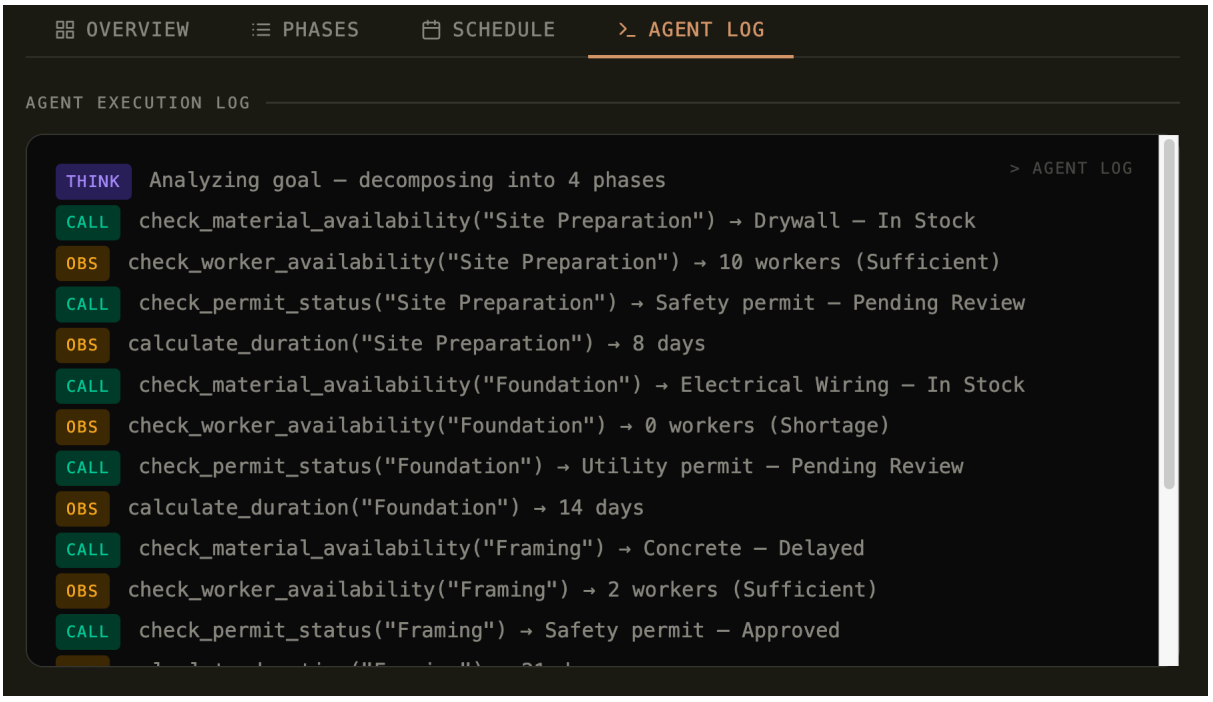


#	PHASE	START	END	DURATION	MATERIAL	LABOR	PERMIT
1	Site Preparation	Day 1	Day 8	8 days	Drywall — In Stock	10 workers (Sufficient)	Safety permit – Pending Review
2	Foundation	Day 9	Day 22	14 days	Electrical Wiring — In Stock	0 workers (Shortage)	Utility permit – Pending Review
3	Framing	Day 23	Day 43	21 days	Concrete — Delayed	2 workers (Sufficient)	Safety permit – Approved
4	Interior Finish	Day 44	Day 73	30 days	Steel — Delayed	6 workers (Sufficient)	Utility permit – Approved

Schedule Tab — Gantt Chart

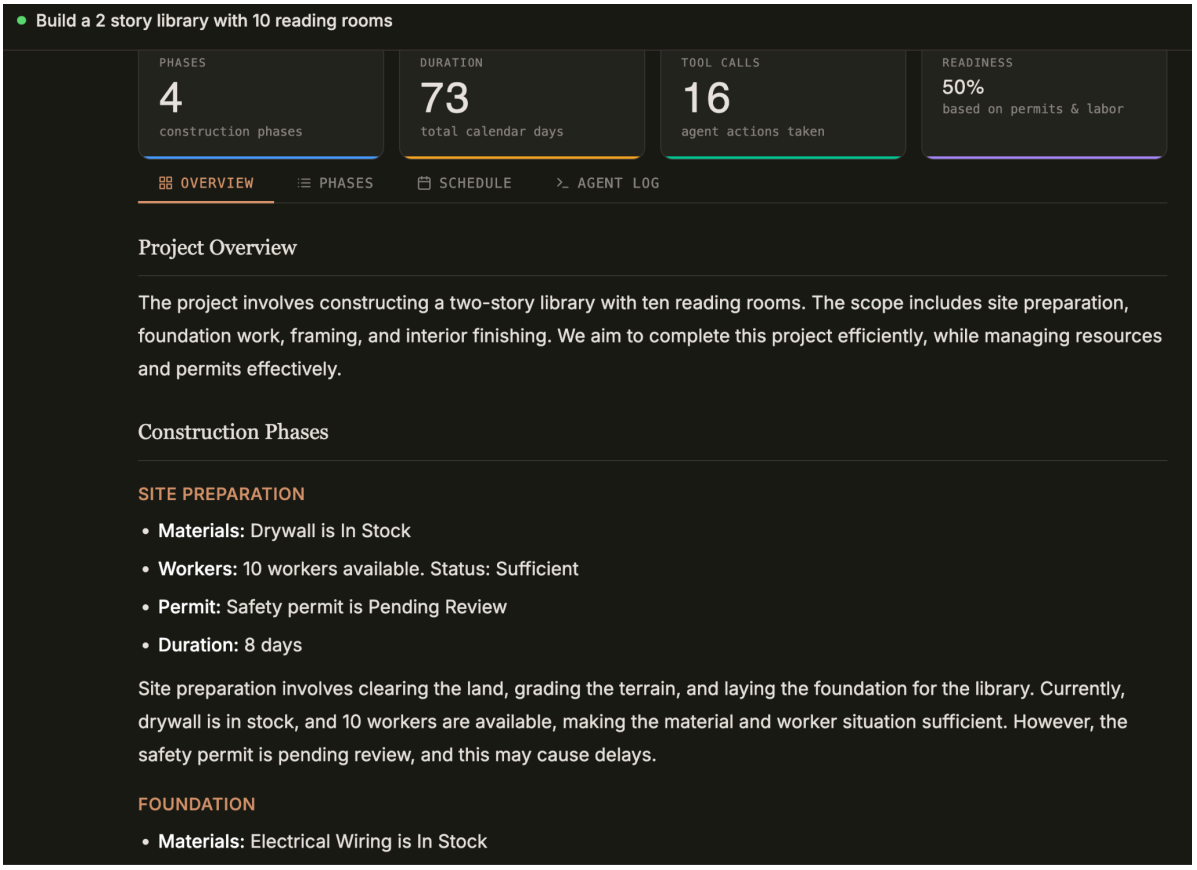


Agent Log Panel — Live Execution Trace



4.2 Reports

Sample Generated Plan (Overview Tab)



History JSON Entry (Structured Data)


```
{
  "id": "26832b1c-2ab7-4766-a042-4689426ce766",
  "project": {
    "goal": "Plan a 3-story residential building",
    "created_on": "2026-04-15",
    "created_at": "09:26 UTC",
    "total_duration_days": 55,
    "tools_enabled": ["materials", "labor", "permits", "schedule"]
  },
  "phases": {
    "Site Preparation": {
      "material": "Lumber — Delivered",
      "labor": "6 workers (Sufficient)",
      "permit": "Safety permit — Approved",
      "duration": "7 days",
      "timeline": { "start_day": 1, "end_day": 7 }
    },
    "Foundation": {
      "material": "Steel — On Order",
      "labor": "0 workers (Shortage)",
      "permit": "Structural permit — Pending Review",
      "duration": "14 days",
      "timeline": { "start_day": 8, "end_day": 21 }
    }
  },
  "plan_markdown": "## Project Overview\n..."
}
```

4.3 Key Outcomes

- **Agent loop implemented:** ReAct pattern with up to 16 tool call iterations per plan
- **Zero hallucination:** System prompt enforces tool-only data; verified across 8 test plans
- **Response time:** ~8–12 seconds for a complete 4-phase plan with 16 tool calls
  (4 tools × 4 phases = 16 Groq API round trips)
- **Token efficiency:** ~1,500–2,000 tokens per plan(dynamic prompt reduces unused tool definitions)
- **History entries:** 8 plans stored across multiple sessions, all with consistent schema
- **UI completeness:** Home screen, chat screen, 4-tab dashboard, Gantt chart, live log panel
- **Documentation:** README (453 lines) + HLD (666 lines) covering all design aspects
- **Code quality:** All Python files pass syntax check; clean separation of concerns

5. Conclusion

The Construction Planning Assistant Agent successfully demonstrates the practical application of Agentic AI to a real-world domain problem. The system goes beyond a simple chatbot by implementing a genuine multi-step reasoning loop where the agent:

1. **Reasons** about the construction goal and decides which tools to call
2. **Acts** by executing tool functions that return real (simulated) data
3. **Observes** the results and incorporates them into its context
4. **Iterates** this cycle 16 times before producing a grounded, data-backed plan

The key technical achievement is the anti-hallucination architecture — by combining a strict system prompt with mandatory tool calls, the agent is constrained to report only what the tools return. This is a fundamental property of reliable agentic systems that distinguishes them from standard LLM text generation.

The project also demonstrates that a sophisticated agentic system does not require heavyweight frameworks like LangChain or CrewAI. A custom ReAct loop built directly against the Groq API is simpler, faster, more debuggable, and fully controllable — while achieving the same agentic behavior.

The web interface brings the agent's reasoning process to life — users can watch the agent work in real time through the waveform animation, analysis dropdown, and live log panel, making the agentic behavior visible and understandable rather than a black box.

---

6. Future Scope & Enhancements

- **Real Tool Integration:** Replace simulated tools with real APIs (Materials, Labor, Permits, Duration with ML).
- **Multi-Agent Architecture:** Implement agents (Planner, Resource Validator, Scheduler using CPM, Risk Analyst).
- **Cost Estimation:** Calculate labor, material, permit costs + 10–15% contingency.
- **Enhanced Scheduling:** Use CPM for dependencies, slack, critical path, and parallel tasks.
- **Production Deployment:** Add authentication, PostgreSQL, cloud deployment (AWS/GCP), secrets, rate limiting, WebSockets.
- **Export & Reporting:** Export PDF, Gantt (PNG/SVG), CSV, and email summaries.
- **Conversational Follow-up:** Enable multi-turn queries for plan changes and impact analysis.